use them in our regular controllers to create and/or send emails. Note that just calling the method we defined isn't enough; we also need to tell Rails to actually send the email. The reason this doesn't happen automatically is that Rails can't be 100 percent sure if you want to deliver the email right this moment, while the user waits, or later, in a background job.

Generally, you don't want the user to have to wait for emails to get sent, because this can take a while. Instead, we'll send it in a background job (which we'll learn more about later in the chapter) by calling `deliver_later` (to send the email right now, you'd use `deliver_now`.[65])

rails7/depot_qa/app/controllers/orders_controller.rb

```
    def create
      @order = Order.new(order_params)
      @order.add_line_items_from_cart(@cart)

      respond_to do |format|
        if @order.save
          Cart.destroy(session[:cart_id])
          session[:cart_id] = nil
»         OrderMailer.received(@order).deliver_later
          format.html { redirect_to store_index_url, notice:
            'Thank you for your order.' }
          format.json { render :show, status: :created,
            location: @order }
        else
          format.html { render :new, status: :unprocessable_entity }
          format.json { render json: @order.errors,
            status: :unprocessable_entity }
        end
      end
    end
```

And we need to update `shipped` as we did for `received`:

rails7/depot_qa/app/mailers/order_mailer.rb

```
  def shipped(order)
    @order = order
```